

PROJECT REPORT

Project Sentinel — Real-Time UPI Transaction & Fraud Detection Pipeline

Submitted By-

Saksham Agarwal

B.Tech Computer Science and Engineering

DIT University, Dehradun

Internship / Project

Data Engineering Internship — Celebal Technologies

Program: Celebal Excellence Internship Program 2026

Domain: Data Engineering

Year: 2026

1. Abstract

Project Sentinel is an end-to-end **UPI transaction analytics and fraud detection pipeline** developed using modern data engineering technologies. The project simulates real-world UPI payment telemetry containing both structural data-quality problems and suspicious transaction behaviour.

The pipeline follows a **Medallion Architecture**, processing data through Landing, Bronze, Silver, and Gold layers. Raw transaction data is generated as JSON records and incrementally processed using **Apache Spark Structured Streaming** and stored using **Delta Lake**.

The Silver layer performs data cleaning, type standardization, Personally Identifiable Information (PII) masking, validation, quarantine of invalid records, and duplicate removal. The Gold layer generates business KPIs and applies an explainable rule-based fraud scoring system using transaction amount, transaction velocity, payee fan-out, shared IP addresses, and unusual transaction hours.

A web-based dashboard is also provided to visualize transaction volumes, fraud alerts, risk scores, data-quality statistics, and business KPIs.

The project demonstrates how a production-style data engineering pipeline can combine **stream processing, data quality, privacy, analytics, and fraud detection** into a single system.

2. Introduction

Digital payment systems such as UPI generate a large volume of transaction data every day. Processing this data reliably requires more than simply storing transactions in a database. Real-world transaction streams can contain malformed records, inconsistent data types, missing values, duplicate transactions, and privacy-sensitive information.

At the same time, fraudulent transactions may appear perfectly valid from a data-format perspective. For example, a transaction can have a valid transaction ID, amount, timestamp, and currency while still exhibiting suspicious behaviour such as:

- An unusually large transaction.
- A large number of transactions within a short period.
- One payer sending money to many different recipients.
- Multiple users operating through the same IP address.
- Transactions occurring during unusual hours.

Project Sentinel addresses both categories of problems:

Data quality problems are handled through validation, transformation, and quarantine.

Behavioural anomalies are handled through explainable fraud-scoring rules.

3. Problem Statement

The objective of the project is to design and implement a scalable transaction-processing pipeline capable of:

1. Generating realistic UPI transaction telemetry.
 2. Introducing realistic data-quality defects.
 3. Ingesting raw transaction data incrementally.
 4. Preserving the original data before transformation.
 5. Structuring raw transaction data.
 6. Cleaning and validating transactions.
 7. Protecting sensitive user information.
 8. Removing duplicate transactions.
 9. Quarantining invalid records with identifiable reasons.
 10. Detecting suspicious transaction behaviour.
 11. Generating business-level transaction KPIs.
 12. Producing explainable fraud alerts.
 13. Presenting results through a dashboard.
 14. Supporting both local execution and Databricks deployment.
-

4. Objectives

The major objectives of Project Sentinel are:

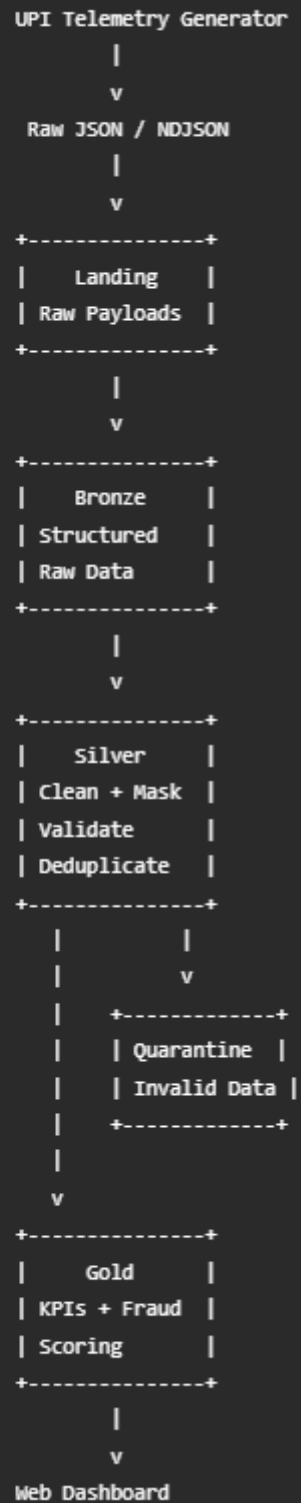
- To implement an end-to-end data engineering pipeline.
 - To understand and implement Medallion Architecture.
 - To process transaction data using PySpark.
 - To use Structured Streaming for incremental ingestion.
 - To store analytical data using Delta Lake.
 - To handle malformed and inconsistent data.
 - To implement data-quality validation.
 - To protect PII using masking and hashing.
 - To identify and remove duplicate transactions.
 - To develop an explainable fraud detection mechanism.
 - To calculate transaction and business KPIs.
 - To create a dashboard for monitoring and analysis.
 - To design the solution so it can be deployed on Databricks.
-

5. Technologies Used

Technology -	Purpose
Python -	Pipeline development and data generation
PySpark -	Distributed data processing
Apache Spark Structured Streaming-	Incremental data ingestion
Delta Lake -	Reliable storage and analytical tables
Databricks -	Cloud execution and deployment target
Unity Catalog -	Data organization and governance
Auto Loader -	Incremental cloud file ingestion
YAML -	Configuration management
React -	Dashboard frontend
TypeScript -	Dashboard development
Vite -	Frontend build and development
Git -	Version control
Pytest -	Testing
Ruff -	Python code quality
Mypy -	Static type checking

6. System Architecture

The project follows a Medallion-style data architecture.



7. Data Generation

The project does not depend on an external UPI dataset. Instead, it contains a custom telemetry generator capable of creating realistic transaction records.

The generator produces transaction attributes such as:

- Transaction ID
- Event timestamp
- Amount
- Currency
- Transaction status
- Transaction type
- Payer bank
- Payee bank
- UPI application
- City
- State
- Merchant category
- Latency
- Payer VPA
- Payee VPA
- Phone number
- Device ID
- IP address

The generator intentionally introduces data-quality problems so that the pipeline can be tested under realistic conditions.

Examples include:

- Missing values.
- Extra whitespace.
- Different casing.
- Amounts represented as strings and numbers.
- Negative amounts.
- Multiple timestamp formats.
- Malformed JSON.
- Duplicate transactions.

The generator also introduces known behavioural anomalies for evaluating the fraud-detection logic.

8. Landing Layer

The Landing layer acts as the immutable raw-data storage layer.

The raw JSON records are read as text rather than immediately parsed into columns.

This design ensures that malformed JSON or unexpected records are preserved instead of being lost during ingestion.

Each landed record contains:

- Original raw payload.
- Source file.
- Landing timestamp.
- Pipeline run ID.

The ingestion process uses **Structured Streaming** with `availableNow`, allowing new files to be processed incrementally while still allowing the pipeline to terminate after processing the currently available data.

9. Bronze Layer

The Bronze layer converts raw JSON payloads into a structured Delta table.

An important design decision is that the original transaction fields remain represented as strings at this stage.

For example:

"842.50"

842.50

-91.2

are preserved instead of immediately being converted into numeric values.

This ensures that the original data can still be inspected when a record fails validation later.

Malformed JSON is also retained instead of being silently discarded.

The Bronze layer additionally records metadata such as:

- Ingestion timestamp.
 - Ingestion run ID.
 - Source file.
 - Ingestion date.
 - Parseability status.
-

10. Silver Layer

The Silver layer is responsible for transforming raw structured data into clean and analytics-ready records.

The major operations are:

10.1 Data Standardization

Whitespace is removed and values are standardized.

Examples:

" Success " → "SUCCESS"

" inr " → "INR"

City and state values are also normalized.

10.2 Type Conversion

The Silver layer converts string representations into appropriate data types.

For example:

"842.50" → 842.50

Transaction timestamps are also converted into a consistent timestamp representation.

The pipeline supports multiple timestamp formats including:

- ISO-8601.
- Epoch seconds.
- Epoch milliseconds.
- dd-MM-yyyy HH:mm:ss.

10.3 PII Protection

Sensitive information is not allowed to flow into analytical layers in its original form.

VPA values are partially masked:

user000012@oksbi

↓

us***@oksbi

Phone numbers retain only their last four digits:

9876543210

↓

XXXXXX3210

Device IDs and IP addresses are converted into salted SHA-256 hashes.

This allows analytical grouping without exposing the original identifiers.

10.4 Data Validation

Records are checked against several validation rules.

Examples include:

- Missing transaction ID.
- Missing timestamp.
- Invalid timestamp.
- Missing amount.
- Invalid amount.
- Non-positive amount.
- Amount exceeding configured limit.
- Invalid transaction status.
- Invalid currency.

10.5 Quarantine

Invalid records are not simply deleted.

Instead, they are written to a quarantine table with:

- Transaction ID.
- Original transaction information.
- Rejection reason.
- Source file.
- Raw payload.
- Rejection timestamp.

This makes the pipeline auditable and allows rejected records to be investigated later.

10.6 Deduplication

Retry storms can cause the same transaction to appear multiple times.

The pipeline uses:

`dropDuplicatesWithinWatermark(transaction_id)`

with a bounded watermark to remove duplicate transaction IDs while preventing unlimited streaming state growth.

11. Gold Layer

The Gold layer contains business-ready information.

It generates:

- Scored transaction fact table.
- Fraud alert table.
- Daily KPI table.
- Dimension-level KPI table.

The Gold layer uses batch processing over the Silver Delta data because the calculations involve complete aggregations and window-based comparisons.

12. Fraud Detection System

Project Sentinel implements an explainable, rule-based fraud scoring mechanism.

The system does not use a machine-learning model. Instead, each suspicious behaviour contributes a predefined score.

Fraud Rules

Rule	Condition	Weight
High Amount Transaction	exceeds robust amount threshold	30
Shared IP	≥5 distinct payers use the same IP	25
Velocity	>10 transactions from a payer within 5 minutes	25
Fan-out	>15 distinct payees within 60 minutes	25
Odd Hour	Transaction between 00:00 and 05:00 IST	10

The final fraud score is the sum of all triggered rules.

Risk bands are then assigned:

Score < Medium Threshold

↓

LOW

Score ≥ Medium Threshold

↓

MEDIUM

Score $\geq$ High Threshold



HIGH

An important design principle is that **no single rule is sufficient to reach the HIGH-risk category**.

This reduces false positives because a single unusual characteristic does not automatically mean that a transaction is fraudulent.

13. Explainable Fraud Alerts

Each alert contains the rules that contributed to its score.

For example:

Transaction: TXN000000201164

Amount: ₹8,453

Fraud Score: 85

Reasons:

- shared_ip
- velocity
- fanout
- odd_hour

This makes the system explainable to an analyst.

Instead of simply reporting:

Fraud = TRUE

the system explains:

Fraud Score = 85

Reasons = shared IP + high velocity + fan-out + odd hour

This is useful for fraud investigation and operational decision-making.

14. High-Amount Threshold Improvement

During development, an important issue was identified with the original high-amount detection strategy.

Using the 99.5th percentile as the threshold caused the threshold to move into the fraudulent transaction population because approximately 0.7% of transactions were deliberately generated as anomalies.

The initial threshold reached approximately:

₹103,899

while injected suspicious transactions started around:

₹60,000

Therefore, many anomalous transactions were missed.

The solution was to use a robust **Tukey fence**:

Upper Fence = $Q3 + 3 \times IQR$

with a minimum domain threshold.

This approach is less affected by a small number of extreme outliers and significantly improved detection performance.

15. Key Performance Indicators

The Gold layer calculates several business metrics.

Daily KPIs

- Transaction count.
- Successful transaction count.
- Total transaction amount.
- Success rate.
- Average transaction amount.
- Distinct payers.
- Number of flagged transactions.
- Amount associated with flagged transactions.

Dimension KPIs

The same metrics can be analyzed by:

- Bank.
- UPI application.
- City.

This allows business users to identify transaction trends and suspicious activity across different dimensions.

16. Project Results

A configured full-scale run produced the following results:

Metric	Result
Raw JSON lines generated	207,464
Records landed	207,464
Records cleansed into Silver	199,629
Records quarantined	4,924
Duplicate transactions removed	2,911
Transactions scored	199,629
HIGH-risk alerts	3,108

The pipeline completed the full run on a laptop in approximately **31 seconds** at the configured scale.

17. Fraud Detection Results

The injected anomalies were used as ground truth for evaluating the detection system.

Anomaly	Planted	Alerted	Recall
High Amount	800	780	1.00
Velocity	1,255	632 HIGH	1.00
Fan-out	1,744	1,107 HIGH	1.00
Odd Hour	600	589	1.00

The evaluation distinguishes between:

- Anomalies that were never scored because they were quarantined or deduplicated.
- Anomalies that reached Gold but were not detected.
- Anomalies that reached the HIGH-risk category.

This separation prevents data-ingestion problems from being hidden inside fraud-detection metrics.

18. Business Results

For the evaluated run, the system produced approximately:

- **₹30.84 crore** in successful transaction volume.
- **88.2%** transaction success rate.
- **₹1,751** average transaction value.
- Approximately **10,100 daily active payers**.

The system also provides KPIs across banks, applications, and cities.

19. Dashboard

A React and TypeScript web dashboard is included with the project.

The dashboard provides:

Overview

Displays:

- Transaction volume.
- Processing statistics.
- Risk-band distribution.

Medallion Funnel

Shows how records move through:

Raw → Landing → Bronze → Silver → Gold

and displays rejected and duplicate records separately.

Business Analytics

Displays:

- Daily transaction volume.
- Flagged transaction percentage.
- Bank-level KPIs.
- Application-level KPIs.
- City-level KPIs.

Fraud Scoring

Displays:

- Score distribution.

- Risk-band thresholds.
- Frequency of individual rules.
- Rule combinations.

Detection Analysis

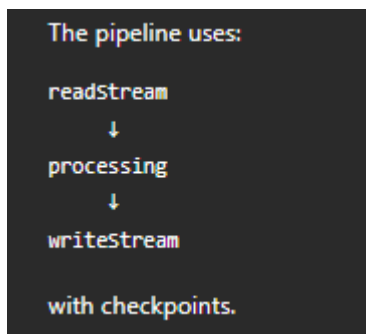
Displays injected anomaly counts and detection results.

Alert Console

Provides a searchable and filterable list of HIGH-risk transactions.

20. Real-Time Processing Design

Although the complete demo can terminate after processing available files, the ingestion architecture uses Structured Streaming.



Trigger.AvailableNow allows the pipeline to process all currently available files incrementally and then stop.

This provides a practical compromise between:

- Continuous streaming.
- Reproducible batch-style execution.

When new transaction files arrive, only newly discovered files are processed.

21. Databricks Integration

The project is designed to run both locally and on Databricks.

The Databricks deployment uses:

- Databricks Runtime.
- Unity Catalog.
- Unity Catalog Volumes.
- Delta tables.

- Auto Loader.
- Databricks Jobs.

The same core pipeline code is used in both environments.

The environment-specific configuration determines whether the pipeline uses local files or Databricks/Unity Catalog resources.

This separation keeps business logic independent of the execution environment.

22. Testing and Code Quality

The project includes automated testing and static analysis.

The reported validation results are:

119 tests passing

Ruff clean

Mypy clean

TypeScript clean

Tests cover areas including:

- Data generation.
- Fraud rules.
- Spark transformations.
- End-to-end processing.
- Deployment configuration.
- Dashboard export.
- Data masking.

This helps ensure that changes to one part of the pipeline do not silently break another component.

23. Security and Privacy

Privacy is an important part of the pipeline.

Sensitive identifiers are protected before reaching the Gold layer.

The project uses:

- Partial VPA masking.
- Phone-number masking.
- Salted SHA-256 hashing for IP addresses.

- Salted hashing for device identifiers.

The Gold layer therefore performs behavioural analysis without exposing raw personal identifiers.

For production Databricks deployment, the project documentation recommends moving the PII salt from the configuration file into a secure secret-management mechanism.

24. Advantages of the System

Project Sentinel provides several advantages:

1. **End-to-end architecture** — covers ingestion, transformation, analytics, and visualization.
 2. **Scalable processing** — uses Spark for distributed processing.
 3. **Incremental ingestion** — uses Structured Streaming.
 4. **Reliable storage** — uses Delta Lake.
 5. **Data-quality handling** — invalid data is identified and quarantined.
 6. **Privacy protection** — sensitive identifiers are masked or hashed.
 7. **Explainable fraud detection** — every alert includes the rules responsible for the score.
 8. **Configurable rules** — thresholds are maintained in YAML configuration.
 9. **Dashboard support** — results can be monitored visually.
 10. **Cloud-ready architecture** — supports Databricks and Unity Catalog.
-

25. Limitations

Despite its capabilities, the project has some limitations.

Rule-Based Detection

The fraud detection system uses predefined rules rather than a machine-learning model. Consequently, it cannot automatically discover previously unknown fraud patterns.

Simulated Data

The transaction data is generated synthetically. Therefore, detection performance does not necessarily represent performance on real UPI transaction data.

Gold Recalculation

The Gold layer is recomputed from Silver data rather than maintaining an incremental state. This is suitable for the project's scale but may require optimization for very large production workloads.

Synthetic Ground Truth

Fraud precision and recall are evaluated against anomalies injected by the generator. This measures agreement with the project's simulated fraud definitions rather than real-world fraud.

Databricks Deployment

The project contains Databricks deployment support, but the repository's documented development run was not executed against a live Databricks workspace.

26. Future Enhancements

Several improvements can be made in future versions.

26.1 Machine Learning-Based Fraud Detection

A machine-learning model could be introduced alongside the rule-based system.

Possible models include:

- Logistic Regression.
- Random Forest.
- XGBoost.
- Isolation Forest.
- Gradient Boosting.

26.2 Real UPI Data Integration

The generator could be replaced or supplemented with anonymized real-world transaction datasets.

26.3 Continuous Streaming

The current AvailableNow approach could be replaced by a continuously running streaming job for real-time monitoring.

26.4 Advanced Alert Management

Future versions could integrate alerts with:

- Email.
- Slack.
- Microsoft Teams.
- Incident-management systems.

26.5 Model Monitoring

If machine learning is introduced, the system could monitor:

- Model precision.
- Recall.
- False-positive rate.
- Data drift.
- Concept drift.

26.6 Production Secret Management

Sensitive configuration values such as the PII hashing salt should be stored using Databricks secret scopes or another enterprise secret-management platform.

Conclusion

Project Sentinel demonstrates the design and implementation of a complete data engineering pipeline for UPI transaction processing and fraud detection.

The project combines **Python, PySpark, Structured Streaming, Delta Lake, Databricks, Medallion Architecture, data-quality processing, PII protection, analytical transformations, and dashboard visualization.**

The pipeline successfully handles both structural and behavioural problems in transaction data. Structural problems are addressed through cleaning, validation, quarantine, and deduplication, while behavioural anomalies are identified through an explainable multi-rule fraud-scoring system.

The project also demonstrates good engineering practices through automated testing, configuration-driven processing, modular code, and separation between pipeline logic and Databricks notebooks.

Overall, Project Sentinel provides practical experience in building a production-style data engineering system from raw transaction generation through data processing, fraud analysis, KPI generation, and visualization.

References

1. [Apache Spark Documentation](#) — Structured Streaming and PySpark.
 2. [Delta Lake Documentation](#) — ACID transactions and lakehouse storage.
 3. [Databricks Documentation](#) — Unity Catalog and Auto Loader.
 4. [Python Documentation](#).
 5. [React and TypeScript Documentation](#).
 6. [Project Sentinel source code and configuration files](#).
-

Project Structure

```
sentinel-upi-main/
├── conf/
│   ├── base.yaml
│   ├── local.yaml
│   └── databricks.yaml
├── databricks/
│   ├── deploy.sh
│   ├── deploy.ps1
│   └── job_sentinel_pipeline.json
├── notebooks/
│   ├── 00_generate.py
│   ├── 01_landing.py
│   ├── 02_bronze.py
│   ├── 03_silver.py
│   └── 04_gold.py
├── src/
│   └── sentinel/
│       ├── generate/
│       ├── landing.py
│       ├── bronze.py
│       ├── silver.py
│       ├── gold.py
│       ├── config.py
│       ├── tables.py
│       ├── schemas.py
│       └── report.py
├── dashboards/
│   └── web/
│       ├── src/
│       ├── package.json
│       └── vite.config.ts
├── tests/
├── run.sh
├── run.ps1
├── Makefile
├── pyproject.toml
└── README.md
```

Final Project Summary

Project Title: Project Sentinel — Real-Time UPI Transaction & Fraud Detection Pipeline

Domain: Data Engineering

Primary Technologies: Python, PySpark, Spark Structured Streaming, Delta Lake, Databricks, Unity Catalog, React, TypeScript

Architecture: Medallion Architecture

Core Features: Data ingestion, data cleaning, validation, PII masking, deduplication, quarantine, fraud scoring, KPI generation, dashboard visualization

Outcome: A complete, modular, testable, and cloud-ready transaction-processing pipeline capable of transforming messy UPI telemetry into clean analytical data and explainable fraud alerts.